

Baryonic Space

FPGA/Digital Design Engineer Interview Project

Responsibilities of the FPGA/Digital Design Engineer:

1. Designing and validating digital logic using Verilog and SystemVerilog
 - Xilinx Vivado based development
 - Basic understanding of the Zynq SoC architecture and DMA
 - Basic understanding of FPGA resources and timing issues
 - Developing comprehensive testbenches in SystemVerilog to verify and validate behavior
2. Translating DSP algorithms implemented in high level languages such as Python, Matlab, and C++ to digital logic
 - Understanding the basic flow of the DSP algorithms developed by colleagues
 - Translating algorithms into digital logic in a generic and extensible fashion
3. Basic understanding of Digital Communications
 - Basic understanding of IQ signals
 - Basic understanding of FFT and windowing
 - Basic understanding of FIR filters
 - Basic understanding of classical digital modulation schemes such as FSK, PSK, and QAM
4. Working with Software Defined Radio frameworks
 - Baryonic Space heavily leverages modern and powerful SDRs to develop innovative solutions in a fast paced and agile manner. The FPGA engineer will deploy the developed DSP cores into the SDR frameworks used by Baryonic Space.

Interview Project

The main goal of this project is to use available Xilinx IP cores to do basic DSP manipulation in digital logic. Please use Verilog for developing the digital logic and SystemVerilog for writing your testbench. Don't use Vivado block design for expressing complex logic.

Part 1: Naive FFT

Task:

Generate Xilinx FFT IP core at 12 bits depth. Use the Vivado generated instantiation code to create an instance in your Verilog module.

In the testbench use a clock of 200MHz. Generate two consecutive complex exponentials of length 256 at center frequencies 12.5 MHz and 10 MHz to investigate the effects of leakage. Compute abs(FFT)^2 in Verilog. Convert the `abs_sqrd_fft` to proper radix and analog format in Vivado waveform viewer and find where the FFT peaks happen using the `xkindex` output of the FFT IP. Verify that the peaks happen at 12.5 and 10 MHz respectively for each input signal.

Goal:

In this part you will explore the leakage effect. It is well known that a sinusoid has an impulse DTFT. However DFT/FFT shows leakage unless the discrete signal is one whole period. See the Part 1 of the attached python notebook to see the effects of leakage.

Tips and tricks:

The following link provides a brief specification to the AXI-4 standard that most digital design intellectual properties (IPs) agree and follow on:
<https://www.scribd.com/document/261640417/IHI0051A-Amba4-Axi4-Stream-v1-0-Protocol-Spec>

Here is a Youtube channel that could be a good guide on designing digital logic and FPGA:
<https://www.youtube.com/@mehmetburakaykenar>

A public repository at your disposal on Github: https://github.com/farbius/dsp_xilinx_ip/tree/main

Part 2: Windowing in Time Domain

Task:

Create a Hamming window in Python (following the guidelines of the attached notebook). Convert the window to the proper fixed-point precision and save the window coefficients in to a txt (.mem) file. In your Verilog module read the coefficients using \$readmemh. Multiply the time domain signal value with the corresponding index of the window coefficients. Supply the windowed time domain signal to the FFT IP. See that leakage behavior is reduced in the waveform viewr.

Goal:

Replicate the effects of windowing as illustrated in the Notebook using HDL.

Part 3: Windowing in Freq Domain

See <https://www.embedded.com/dsp-tricks-frequency-domain-windowing/> for some discussion. (Above link is from the “Frequency-Domain Windowing” section of the Understanding Digital Signal Processing textbook by Richard Lyons)

Effect of windowed FFT can be describe using a three-term effect of the unwindowed FFT.

Equation 13-11

$$\begin{aligned} X_{\text{three-term}}(m) &= \alpha X(m) - \frac{\beta}{2} X(m-1) - \frac{\beta}{2} X(m+1) \\ &= \alpha(a_m + jb_m) - \frac{\beta}{2}(a_{-1} + jb_{-1}) - \frac{\beta}{2}(a_{+1} + jb_{+1}) \\ &= \alpha a_m - \frac{\beta}{2}(a_{-1} + a_{+1}) + j[\alpha b_m - \frac{\beta}{2}(b_{-1} + b_{+1})]. \end{aligned}$$

Where $\alpha = 0.54$ and $\beta = 0.46$

In this part you will apply the three term windowing effect in Verilog. Apply the three-term effect to the output of the FFT IP core.

Goal:

Show that windowing in time domain and windowing in frequency domain using Verilog and proper testbenches. As demonstrated in the attached notebook, the exact same output should be acquired using your three-term Verilog implementation.

Part 4: Zynq PL/PS communication (Bonus)

Add a DDS component to your design to simulate a complex sinusoid. Your windowed-FFT core should find the index where the FFT peaks. It should be possible to do the following from PS side:

- Read the index where the FFT peaks
- Set the DDS center frequency

Please provide the C or C++ code that will run on PS.

Write a simple C or C++ assertion test that checks that the DDS frequency that is set is within the frequency resolution of the index that is estimated via the FFT core.